El Lenguaje



Biblioteca iostream

Biblioteca iostream

- *En C++ se tiende al uso de clases stream para entrada/salida, en lugar de las funciones printf(), fprintf(), scanf() y fscanf().*
- *Una de las razones es que se cometen menos errores con las clases stream ya que solas contemplan el tipo de dato que deben exhibir, sin necesidad de caracteres de formato (%d, %f, etc.)*

Operador ">>"

- *El operador de extracción ">>" toma dos operandos. El de la izquierda es una expresión stream (ej. cin) , el operando de la derecha es una variable simple.*
- *O sea que extrae el próximo item de la corriente de entrada y almacena su valor en la variable que es operando de la derecha.*

Operador ">>"

- *Ejemplo:*

```
int base, altura;
cin >> base;
cin >> altura;
// es lo mismo que escribir:
cin >> base >> altura;
```

- *Salta todos los blancos. Se entiende por blancos a los espacios, tabs y nuevas líneas.*

cin / cout

```
#include <iostream>
```



Operador "<<"

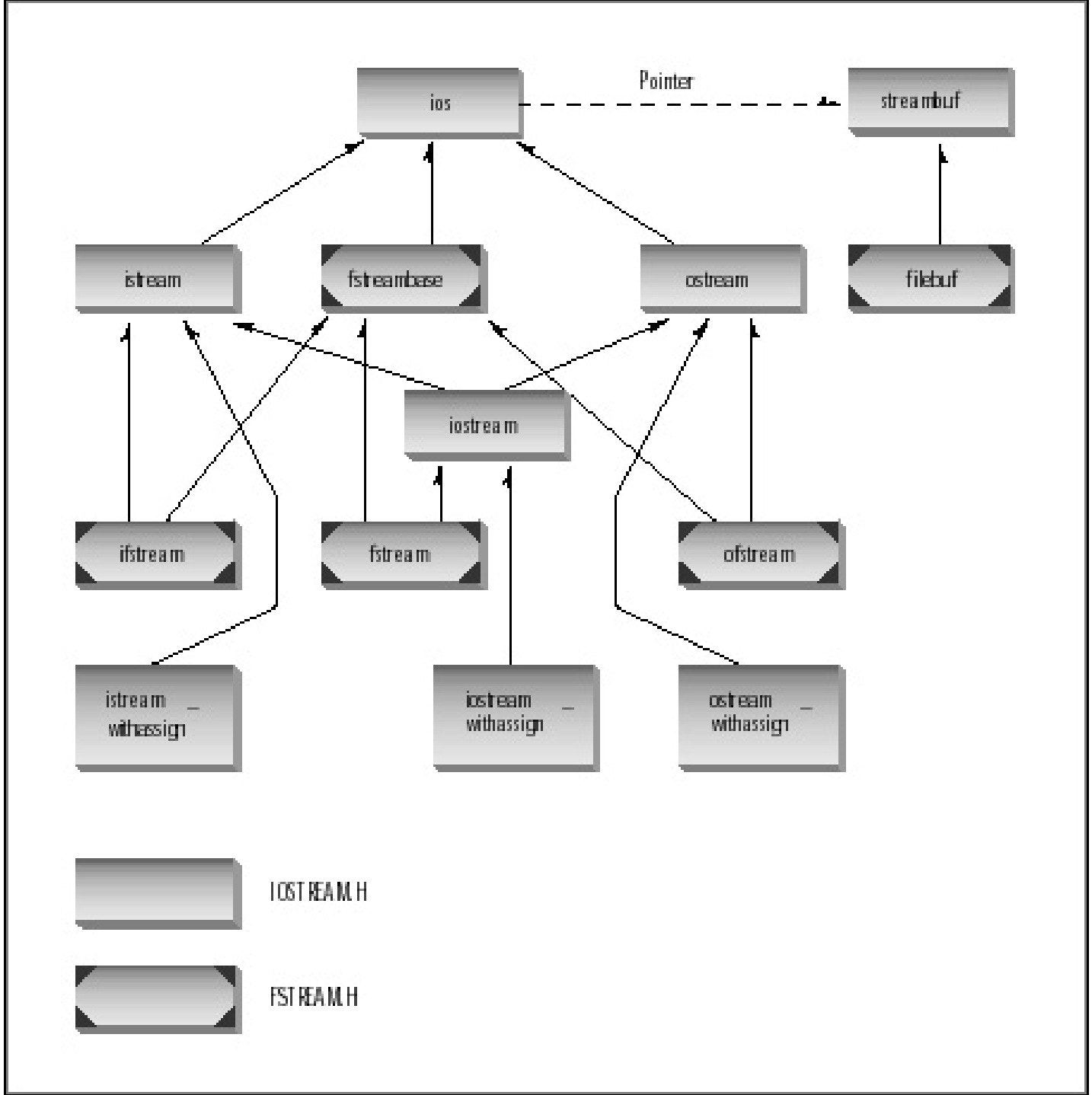
- *El operador de inserción "<<" toma dos operandos. El de la izquierda es una expresión stream (ej. cout) , el operando de la derecha es una variable simple o constante.*
- *O sea que inserta el contenido de la variable o constante que está a derecha en la corriente de la izquierda, que no irá a pantalla hasta una operación flush.*

Biblioteca iostream

- *Se puede recargar los operadores de inserción "<<" y de extracción ">>", para trabajar con las clases que nosotros generemos.*
- *Esto sucede no solo para la pantalla sino también para archivos y para dar formato a datos en memoria, con el propósito de usarlos en cajas de diálogo para programación gráfica.*

Biblioteca iostream

- *Las clases stream están organizadas en una jerarquía.*
- *El operador de inserción "<<" es miembro de la clase **istream** y el operador de extracción ">>" es miembro de la clase **ostream**.*
- *Estas dos clases (istream y ostream) derivan de **ios**.*


ios
Pointer
streambuf
istream
fstreambase
ostream
filebuf
iostream
ifstream
fstream
ofstream
istream_withassign
iostream_withassign
ostream_withassign
IOSTREAM.H
FSTREAM.H

cin / cout

- *El objeto **cout**, representa la salida estándar, que se dirige a pantalla, es un objeto predefinido de la clase:*

  ***ostream_withassign**, que deriva de la clase ostream.*
- ***cin** es un objeto predefinido de la clase:*
- ***istream_withassign**, que deriva de la clase istream.*

Biblioteca iostream

- *Las clases que trabajan con el monitor y teclado son declaradas en el archivo de encabezamientos "iostream".*
- *Las que lo hacen con archivos están declaradas en "fstream".*
- *Los manipuladores de stream están declarados en "iomanip".*
- *Las clases in-memory están declara-das en "strestrea"*

Biblioteca iostream

- *Como se puede observar el ancestro de todos es:* ***ios****.*
- *Esta clase contiene un puntero a la clase streambuff, que contiene el buffer de memoria donde los datos son escritos y leidos y las rutinas de bajo nivel para manejarlos.*
- *Esta clase es referenciada automáticamente por las otras de la jerarquía.*

Otra forma de operación

- *Las clases istream y ostream permiten trabajar con la entrada y salida de una forma totalmente distinta.*
- *Este mecanismo consiste en utilizar métodos de los objetos stream, como cualquier método de un objeto común.*

Clase istream

- *La clase istream contiene los métodos get(), getline(), read().*

```
cin.get(char &c);
// ingresa un caracter solo, sin saltar blancos
cin.get(char *s, int n, char c = '\n');
// ingresa stream en s hasta '\n' o n-1 caracteres
cin.getline(char *s, int n, char c = '\n');
// ingresa stream en s hasta '\n' o n-1 caracteres
// excluye el delimitador.
cin.read(char *s, int n);
// ingresa stream en s hasta n-1 caracteres (acti-
// va excepción si encuentra EOF antes de n chars)
```

Clase ostream

- *La clase ostream contiene los métodos put() y write().*

```
cout.put(char c);
// Salida de char simple
cout.write(char *s + int m, int n);
// escribe n elementos del string s después de
// saltar m elementos
cout.flush(); // vacía el buffer a la corriente
```

- *La clase iostream deriva de istream y de ostream por herencia múltiple*

iostream

- *Las clases derivadas de la clase iostream pueden ser usadas con dispositivos tales como disco, que pueden ser abiertos para lectura/escritura simultáneas.*
- *Esto lo hacen las clases:*
  - *istream_withassign (istream)*
  - *ostream_withassign (ostream)*
  - *iostream_withassign (iostream)*

ios

- *En la clase **ios** encontramos la mayo-ría de las características necesarias para trabajar con C++.*
- *Estan son:*
  - *Banderas de formato.*
    - *Manipuladores*
  - *Bits de error-estado*
  - *Modos de operación de archivos*

Banderas de formato

- *Las banderas son métodos de **ios**, actúan como estados (on/off) especificando varios aspectos del formato de entrada/salida.*
- *Por ser miembros de la clase ios, deben ir generalmente precedidos por ios::, o sea el nombre de la clase y el operador de resolución de ámbito.*

Banderas de formato

- *Hay varias formas de utilización de las banderas de formato.*
- *Además diferentes banderas pueden ser activadas de formas diferentes.*
- *Todas las banderas pueden ser activadas y desactivadas mediante las funciones miembro*

setf() y unsetf()

Ej.: cout.setf(ios::left)

Banderas de formato

skipws	*Saltar espacios en la entrada.*
left	*Ajustar a izquierda [12.34].*
right	*Ajustar a derecha [12.34].*
internal	*Usar relleno entre el signo y el número [+12.34].*
dec	*Convertir a decimal.*
oct	*Convertir a octal.*
hex	*Convertir a hexadecimal.*
showbase	*Usar indicador de base en la salida (0, 0x).*
showpoint	*Muestre el punto decimal en la salida.*
uppercase	*Usar mayúsculas (también en salida hexa)*
showpos	*Mostrar '+' delante de enteros positivos.*
scientific	*Usar formato exponencial [9.1234E2].*
fixed	*Usar formato fijo [912.34].*
unitbuf	*Descargar todos los streams después de inserción.*
stdio	*Descargar stdout, stderror después de inserción*

Manipuladores

- *Los manipuladores son instrucciones de formato insertadas directamente en el stream.*
- *Existen dos categorías, los que llevan argumento y los que no.*
- *Los que no llevan argumento permanecen vigentes hasta el fin del stream.*
- *Los que llevan argumento pueden afectar solamente al elemento siguiente.*

Manipuladores

Sin argumento

`ws`	*Habilitar salto de espacios en blanco en la entrada.*
`dec`	*Convertir a decimal.*
`oct`	*Convertir a octal.*
`hex`	*Convertir a hexadecimal.*
`endl`	*Insertar nueva línea y vaciar stream de salida.*
`ends`	*Insertar carácter null para terminar la salida.*
`flush`	*Vaciar el stream de salida.*
`lock`	*Bloquear el manipulador de archivo.*
`unlock`	*Desloquear el manipulador de archivo.*

Manipuladores

Con argumento

`setw()`	*ancho del campo (int)*
`setfill()`	*char con el que se llena blancos (int).*
`setprecision()`	*precisión (int) .*
`setiosflags()`	*banderas de formato (long).*
`resetiosflags()`	*banderas de formato (long) (reset)*
`setbase()`	*selecciona base numérica (int)*

Objetos stream predefinidos

cin	***Entrada por teclado.***
cout	***Salida por pantalla.***
cerr	***Salida de diagnóstico.***
clog	***Salida de bitácora (log).***

- *Es interesante revisar con cuidado la biblioteca ios cuando debamos utilizar características especiales del manejo de entrada/salida.*
- *Dispone de muchas alternativas que normalmente no son recordadas.*

Manejo de Archivos

Archivos en C++

- *A diferencia de C, C++ provee mecanismos más simples para la manipulación de archivos, esto es por el uso de objetos que nos facilitan la tarea.*
- *Para toda operación con archivos se debe incluir <fstream>, si bien esto automáticamente incluye <iostream>, es conveniente hacerlo en forma explícita para usar cin y cout.*

Archivos en C++

- *Para abrir un archivo para lectura creamos un objeto ifstream, que luego se comporta como cin.*
- *Para abrir un archivo para escritura creamos un objeto ofstream, que luego se comporta como cout.*
- *Una vez abierto el archivo se lee y se escribe igual que cuando tratamos con cualquier objeto iostream.*

```
#include <iostream>  // P06-07.cpp
#include <fstream>

using namespace std;

int main( void )
{
  char linea[80];
  ofstream fp;

  fp.open( "miarchivo.txt", ios::out );
  if( !fp ) {
    cout << "ERROR en apertura de archivo" << endl;
    exit(1);
  }                 // traslada entrada por teclado al archivo
  while(1) {        // hasta que ingresamos "." en una línea.
    cin.getline( linea, 80, '\n' );
    if( (strlen(linea)==1) && (linea[0]=='.') )
      break;
    fp << linea << endl;
  }
  fp.close();
  return 0;
}
```

Archivos en C++

- *Modos de actualización en acceso directo:*

app	***Abrir el archivo para agregado***
ate	***Buscar el fin de archivo después de la apertura***
in	***Abrir el archivo para lectura***
out	***Abrir el archivo para escritura***
binary	***Abrir el archivo en modo binario***
trunc	***Descartar el contenido del archivo si existe***
nocreate	***La apertura debe fallar si el archivo no existe***
noreplace	***Si el archivo existe, la apertura debe fallar a menos que se agregue o se busque fin de archivo***

Archivos en C++

- *Equivalencias de lugar de inicio de búsqueda entre C y C++ para archivos aleatorios.*

Descripción	En C	En C++
Comienzo de archivo	SEEK_SET	ios::beg
Posición actual	SEEK_CUR	ios::cur
Fin de archivo	SEEK_END	ios::end

Manejo de Excepciones

Excepciones en C++

- *A diferencia de C, C++ tiene un mecanismo para el manejo de lo inesperado, denominado: Manejo de Excepciones.*
- *Significa que una función le dice a su llamador que no puede hacer lo que se le está solicitando.*
- *La forma que realiza C esto, es por el retorno de código de error, pero estos puede ser ignorados, la excepción NO.*

Excepciones en C++

- *Las excepciones nos permiten construir funciones puras que, o terminan exitosmente retornando un valor válido o fallan y terminan a través de una excepción.*
- *Esto último no es menor y es fundamental para cualquier manejo de errores.*
- *En C++ la terminación fallida se denomina: Arrojar una Excepción.*

```
#include <iostream>  // P06-10.cpp

using namespace std;

int divide( int dividend, int divisor ) throw ( const char* );

// Divide el dividendo con el  divisor que debe ser !=0

int main( void ) {
  try {
    int result = divide( 50, 2 );
    cout << "División(" <<50<< ", " <<2<< ") da " << result << endl;
    result = divide( 50, 0 );
    cout << "divide(" <<50<< ", " <<0<< ") da " << result << endl;
  }
  catch (const char* msg) {
    cout << "Epa, gran error!: " << msg << endl;
  }
  return 0;
}
```

```
División(50, 2) da 25
Epa, gran error!: Se intentó una división por cero
```

```
int divide( int dividend, int divisor ) throw ( const char* ) {
  if (divisor == 0)
    throw (const char*)"Se intentó una división por cero";
    // No debemos preocuparnos por una división por cero
  return dividend/divisor;
}
```

Excepciones en C++

- *En el programa anterior, al prototipo de la función "divide" se agrega una especificación de excepción:*

```
throw (const char*)
```

- *Las excepciones tienen tipo y una función puede arrojar excepciones de distintos tipos.*
- *Estas se muestran como una lista separada por comas.*

Excepciones en C++

- *En el ejemplo solamente se utilizó una constante del tipo char "const char*".*
- *Los intentos por capturar algo que no debería suceder, deben encerrarse en bloques try/catch.*
- *Hemos visto un bloque try seguido de uno catch, pero puede haber varios catch.*

Excepciones en C++

- *Si alguna operación dentro del bloque try arroja una excepción (en nuestro caso "divide"), la ejecución abandona el bloque try y ejecuta el bloque catch con el mismo tipo que arrojó la excepción.*
- *En nuestro caso al llamar divide con denominador 0, abandona el bloque try y entra en el bloque catch con el mismo tipo de la excepción arrojada.*

Excepciones en C++

- *Si no se encuentra un bloque catch que corresponda al tipo de excepción arrojada, la ejecución deja la función y trata de ubicar un bloque catch en el llamador.*
- *Si no encuentra un bloque catch cuando se alcanza "main", termina la ejecución del programa.*
- *La cantidad de información suministrada por la excepción se puede mejorar.*

Excepciones en C++

- *La mayor efectividad en el manejo de excepciones viene cuando esta se incorpora desde un principio.*
- *Como las excepciones tienen un tipo y como esto no es muy informativo, se puede crear una estructura que nos brinde más información.*
- *Los distintos tipos ahora están dentro de la estructura.*

Namespaces

Evitando conflictos de nombres

- *Namespaces fue introducido en el estándar de 1995.*
- *Su principal función es evitar conflictos de nombres en proyectos de gran escala, además de facilitar el manejo de configuraciones y control de versiones.*
- *Los conflictos de nombre surgen siempre que el espacio en que existen es plano.*

Namespaces

- *Usar namespaces en C++ es equivalente a usar una jerarquía de directorios en un sistema de archivos.*
- *O equivalente al sistema de nombres de dominio en Internet (DNS).*
- *Es decir permite generar espacios diferentes que puede ser vinculados en forma jerárquica.*
- *El namespace por defecto es:* ***std***

Namespaces

- *A principios de los ´90 cuando C++ comenzó a popularizarse era común que se proveyera paquetes de clases propietarias por parte de muchas firmas.*
- *En muchos casos estas clases se llamaban igual y era imposible poder distinguirlas para un compilador .*
- *Observar que estamos hablando de clases con nombres iguales, no métodos.*

Namespaces

- *Sería un error pensar que esto sucede solamente cuando utilizamos software de terceros.*
- *Los conflictos de nombre también existen en grandes proyectos, aunque todo haya sido desarrollado por el mismo conjunto de desarrolladores.*
- *Antes de disponer de namespaces los conflictos solían solucionarse así:*

Namespaces

```
class string { // nombre corto y quizás muy repetitivo
    //...
};
class XCompany_string { // nombre que seguramente no
    //...                 Colisionará, pero incómodo.
};
```

- *Imaginamos lo que sucede si la compañía cambia de nombre.*
- *Es evidente que la solución en el mejor de los casos es un parche.*
- *Afortunadamente con namespaces podemos usar nombre cortos sin conflicto.*

Namespaces

- *Los nombre cortos y significativos que pretendemos usar se pueden agrupar bajo un namaspace así:*

```
// archivo XCompany.h
namespace XCompany { // definición de un namespace
    class string { /*...*/ };
    class vector { /*...*/ };
};
```

- *Los miembros de namespaces pueden definirse fuera de su declaración, igual que sucede con los métodos.*

Namespaces

```
#include <iostream>

using namespace std;

namespace A {
  void f();     // declaración
}

void A::f() {   // definición en lugar separado
  cout << "en f" << endl;
}

int main() {
  A::f();
  return 0;
}
```

Namespaces

- *Mediante la directiva "using" se define el espacio de nombre por defecto.*

```
using namespace std;
```

- *A partir de aquí todo nombre que indiquemos pertenece a ese espacio de nombres.*
- *Esto nos permite la utilización directa de cin y cout sin preasociarles "std::"*

Namespaces

- *Si esto no fuera así, el famoso "hello world" sería:*

```
#include <iostream>
int main () {
  std::cout << "Hello, world!" << std::endl;
  return 0;
}
```

- *Para no utilizarlo hacemos:*

```
#include <iostream>
using namespace std;
int main () {
  cout << "Hello, world!" << endl;
  return 0;
}
```

using / using namespace

- *using es tanto una declaración como una directiva.*
- *En su papel de declaración, indicamos que todas las referencias primarias (no calificadas con nombre) a vector corresponden al espacio de nombres indicado*

```
#include <vector>
int main() {
  using std::vector;  // cada ocurrencia de vector
  vector vi;          // será buscada en std
  return 0;
}
```

using / using namespace

- *En su papel de directiva indica el espacio de nombres por defecto.*

```
#include <vector>
#include <iostream>

int main() {
  using namespace std; // todo será buscado en
  vector vi;           // std, no solamente vector
  vi.push_back(10);
  cout << vi[0];
  return 0;
}
```

using / using namespace

- *Para usar ambas alternativas en un programa, haríamos:*

```
#include <string> //  std::string
#include "XCompany.h"

int main() {
  using namespace XCompany;
  string s; // se refiere a la clase XCompany::string
  std::string standardstr; // se refiere al string
                           // estándar
  return 0;
}
```